REPORT · 2026年8月

造像类文物三维建模自动化系统 先导底座代码说明书

P0 核心模块 · 配置 / 日志 / 溯源 / 异常

文保数字化研发组

2026年8月

一、概述

本文档是《造像类文物三维建模自动化系统（路线 E）》先导底座（P0 核心模块）的代码说明书。先导底座对应工程目录 src/h3d/core 子包，是整条三维建模自动化管线的全局基础设施：配置校验、分级日志、溯源元数据与自定义异常。它不包含任何点云 / 网格算法，但决定了后续所有业务代码「如何被配置、如何被记录、如何被追踪、如何报错」。本说明书严格遵循《软件开发项目设计书》第 31 章（M0 详细设计）、《管线骨架构建分析》第 4~5 章（模块划分与依赖约束）及《开发指南（教学版）》第 12、13、28 章的接口约定。

> 设计铁律：core 底座不得反向依赖任何业务子包（io / preprocess / register / change / colour / runner / report），保证底座可被单独测试、无循环导入。这是「概念 → 目录」投影一致性的第一道守门员。

二、模块划分

先导底座由 4 个业务模块与 2 个包导出文件构成，目录结构如下：

TEXT

```
src/h3d/core/
■■■ __init__.py      # ■■■■■■■■■■■■■__all__■
■■■ exceptions.py    # ■■■■■■■■■■■■■■■■■■■■
■■■ config.py        # dataclass + ■■■■■■■■■■■■■■ YAML ■■
■■■ logging.py       # ■■■■■ DEBUG / INFO / WARNING / ERROR
■■■ metadata.py      # SHA256 ■■ + run_manifest ■■■■■■
src/h3d/__init__.py # ■■■■■■■ __version__ = "0.1.0"
```

模块文件	核心职责	对外关键符号
exceptions.py	管线自定义异常层级（错误码 E001~E900）	H3DError, RecoverableError, ConfigError, InputDataError, RegistrationError, QualityGateError, ExportError, ExternalProcessError

config.py	参数校验 + YAML 解析 + 序列化	Field, Nested, PipelineConfig, ProjectConfig, PreprocessConfig, RegisterConfig, ChangeConfig, ColourConfig, RunnerConfig, load, loads, dump
logging.py	分级日志系统（控制台 + 文件双 handler）	setup_logging, get_logger
metadata.py	输入指纹 + 溯源元数据 + 断点续跑	sha256_file, sha256_bytes, sha256_text, generate_run_id, software_versions, RunManifest

四个模块的职责边界清晰：exceptions 只管「错误如何表达与分级」，config 只管「参数如何声明与校验」，logging 只管「过程如何记录」，metadata 只管「产物如何溯源」。四者各司其职、互不越界，通过显式导入耦合。

三、依赖关系与数据流

3.1 包内依赖方向

依据《管线骨架构建分析》5.1，core 包内依赖方向自上而下、禁止反向环，其中异常为最底层：

TEXT

```
core package internal deps (A -> B means A imports B):
  config.py    -> exceptions.py     [validation failures -> ConfigError]
  logging.py   -> (none)            [stdlib logging only]
  metadata.py  -> (none)            [stdlib hashlib/json only]
  __init__.py  -> [config, exceptions, logging, metadata]   [aggregate exports]

Package-level deps (base <-> business):
  io / preprocess / register / runner / ...  -->  core
  core  -->  (NONE; does not depend on any business sub-package)
```

> exceptions 是唯一被 config 依赖的模块，而它自身零依赖；logging 与 metadata 仅依赖标准库。这一「单向下沉」结构确保了：单独 import h3d.core.exceptions 不会触发任何外部库加载，底座可独立单测。

3.2 运行时数据流

一次典型运行的底座数据流为：加载 YAML → 描述符校验构造 PipelineConfig → 初始化分级日志 → 编排器逐阶段调用业务模块 → 登记 RunManifest 阶段状态 → 落盘 run_manifest.json。下图给出底座内部的完整流转（自顶向下）：

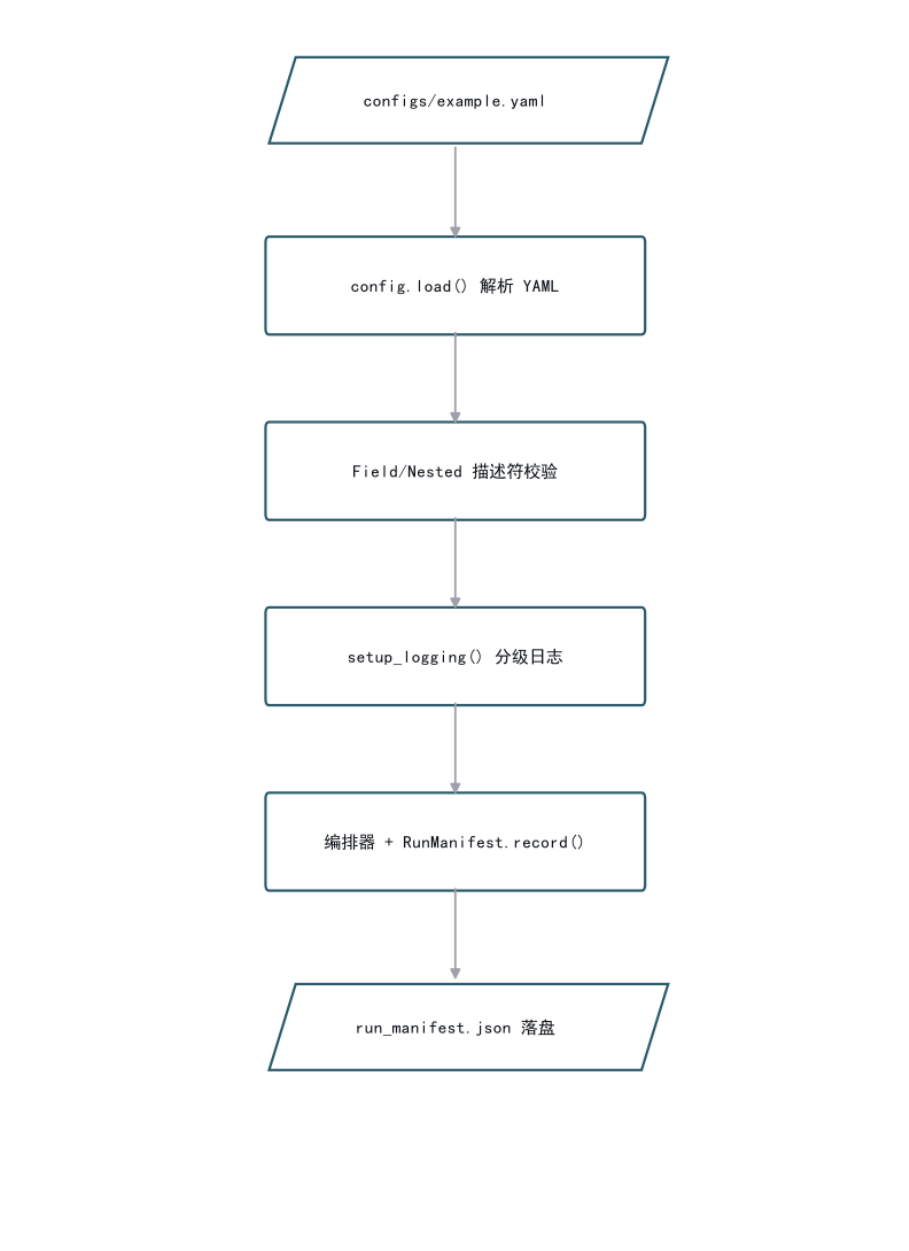


Figure 1: 先导底座运行时数据流（自顶向下；非法取值经 ConfigError 分支中断，见第五章错误码）

四、模块详细设计

4.1 exceptions.py — 异常层级

所有异常继承自基类 H3DError，统一携带错误码 code、阶段 stage、尊像 item_id 三个上下文字段。关键设计是区分「可恢复」与「不可恢复」：RecoverableError 及其子类对应编排层「失败隔离」语义——单尊失败仅跳过该尊（记入 manifest 后继续队列）；ConfigError 与 ExternalProcessError 为不可恢复，按设计书 31.4 直接中断或走回滚路径。

异常类	错误码	可恢复	触发场景
H3DError	E000	基类	所有管线异常的公共基类
RecoverableError	E000	基类	可恢复错误基类（失败隔离判定依据）
ConfigError	E001	否	配置缺失 / 字段非法 / 取值越界
InputDataError	E100	是	原始数据缺失 / 损坏 / 空点云
RegistrationError	E200	是	重叠不足 / RANSAC 失败 / 闭环超差
QualityGateError	E500	是	中误差 / 覆盖率未过质量门
ExportError	E601	是	导出格式不支持 / 路径不可写
ExternalProcessError	E900	否	外部子进程 / 磁盘写失败

PYTHON

```
class H3DError(Exception):
    code: str = "E000"
    def __init__(self, message="", *, item_id=None, stage=None, code=None):
        self.item_id = item_id
        self.stage = stage
        if code is not None:
            self.code = code
        super().__init__(self._format(message))

class RecoverableError(H3DError):
    """Recoverable base: single-item failure is isolated, queue continues."""

class ConfigError(H3DError):
    code = "E001"

class InputDataError(RecoverableError):
    code = "E100"   # similarly: E200/E500/E601/E900
```

4.2 config.py — 配置校验与解析

config 模块采用「dataclass + 描述符元编程」双重机制：描述符 Field 在 __set__ 阶段完成类型 / 取值校验，任何 cfg.attr = v 赋值或构造期赋值都会被拦截；Nested 描述符承载子配置对象实现层级化。所有配置类继承 _ConfigBase，由 __init_subclass__ 扫描描述符声明，自动派生 __init__ / to_dict / from_dict，避免逐类手写样板。YAML 解析统一使用 yaml.safe_load，杜绝代码注入。

配置段	字段（默认值）	对应管线阶段
ProjectConfig	name("") / operator("") / raw_dir(data/raw) / formats([ply,pcd,obj])	项目元信息
PreprocessConfig	voxel_down_sample(0.005) / remove_outlier_std(2.0) / normal_estimate_k(30)	M1 预处理
RegisterConfig	fpfh_radius(0.02) / ransac_distance(0.05) / icp_distance(0.02) / max_iter(2000)	M2 配准
ChangeConfig	enabled(false) / core_radius(0.05)	X1 多时相（可选）
ColourConfig	enabled(false)	X2 色差（可选）
RunnerConfig	fail_isolation(true) / resume(true) / manifest(output/run_manifest.json)	M0 编排

PYTHON

```
class Field(_Descriptor):
    def __init__(self, *, dtype=None, default=_REQUIRED,
                 low=None, high=None, choices=None, nullable=False): ...
    def validate(self, value):
        # 1) type coercion  2) range [low, high] check  3) choices check
        # any failure -> raise ConfigError(...)

class PreprocessConfig(_ConfigBase):
    voxel_down_sample: float = Field(dtype=float, default=0.005, low=0.0)
    remove_outlier_std: float = Field(dtype=float, default=2.0, low=0.0)
    normal_estimate_k: int = Field(dtype=int, default=30, low=1)

def load(path: str) -> PipelineConfig:
    data = yaml.safe_load(open(path, encoding="utf-8"))
    return PipelineConfig.from_dict(data or {})
```

4.3 logging.py — 分级日志

logging 模块提供分级日志（DEBUG / INFO / WARNING / ERROR / CRITICAL），统一格式「时间 [级别] [模块名] 消息」。setup_logging 一次性配置控制台 + 可选文件双 handler，通过「是否存在 handler」判断实现幂等；get_logger 返回带 h3d 前缀的子 logger。日志文件统一 UTF-8 编码，规避中文路径乱码。

PYTHON

```
setup_logging(level="INFO", log_file="output/pipeline.log")
log = get_logger("core.config")   # -> h3d.core.config
log.info("Loaded config: %s", path)
log.warning("Station overlap insufficient")
log.error("Processing failed: %s", err)
```

4.4 metadata.py — 溯源元数据

metadata 模块提供两类能力：一是输入指纹 sha256_file（分块流式，适配大点云文件）、sha256_text、sha256_bytes；二是溯源核心 RunManifest，按设计书 41.1 的字段字典记录每次运行的唯一标识、配置哈希、各阶段状态与软件版本快照，是「可复现 / 可审计 / 可软著溯源」的真相来源，也是断点续跑的判据。

字段 / 方法	类型 / 签名	含义
run_id	string	运行唯一标识，形如 20260824-rock001-a1b2
created_at	datetime	创建时间（ISO 8601）

config_sha256	string	配置哈希（可复现）
stages	object	各阶段状态字典（M1~M6 / X1 / X2）
software	object	依赖版本快照（open3d / numpy / ...）
record(stage, ...)	method	登记阶段结果：status/outputs/elapsed_s/关键指标
record_fail(stage, reason)	method	登记阶段失败原因
is_done(stage)	method	断点续跑判据
save(path) / load(path)	method	序列化落盘 / 恢复

PYTHON

```
m = RunManifest(config_sha256=sha256_file("configs/example.yaml"))
m.record("M1", outputs=["10_clean/a.ply"], elapsed_s=1.5, rmse_mm=1.8)
m.record_fail("M2", "overlap < threshold")
m.save("output/run_manifest.json")   # ensure_ascii=False, indent=2
```

4.5 __init__.py — 包导出

core/__init__.py 聚合导出四模块的公共符号，业务方只需 from h3d.core import ... 即可。顶层 src/h3d/__init__.py 声明 __version__ = "0.1.0"，供 CLI 与报表层引用版本号。

五、接口契约与错误码

底座对外暴露的稳定接口遵循设计书 13.2 的「函数 + 配置 + 目录契约」约定。错误码与设计书 13.3 严格对齐：

错误码	含义	处理策略
E001	配置缺失 / 非法	启动即报错退出（不可恢复）
E100	输入缺失 / 损坏	标记跳过，记入 manifest
E200	配准失败	回退粗配准或告警
E500	质检不达标	阻断交付，提示重跑参数
E601	导出格式不支持	跳过该格式，记入 manifest

E900	外部子进程 / 磁盘异常	隔离，不影响队列其余项

编排层（M0）通过 except RecoverableError 捕获可恢复错误，实现失败隔离；「先记 manifest 再落盘」的顺序由 RunManifest.record 的调用约定保证，中断后可恢复。

六、测试与验证

先导底座配套单元测试位于 tests/test_core/，覆盖设计书第 31.5 节的 M0 测试用例与哈希 / 日志边界，共 29 项，全部通过。

测试文件	覆盖点
test_exceptions.py	异常层级继承、错误码、消息格式化、可恢复性判定
test_config.py	默认值、YAML 解析、描述符越界 / 未知字段报错、序列化往返、类型转换
test_metadata.py	SHA256 已知向量、RunManifest 断点续跑、save/load 往返
test_logging.py	级别解析、命名空间、幂等配置（不重复叠加 handler）

BASH

```
$ python -m pytest tests/test_core -q
.............................           [100%]
29 passed in 0.31s
```

> 运行方式：pip install -e ".[dev]" 安装本包后执行 pytest；或直接 python -c "from h3d.core import PipelineConfig, load" 进行冒烟验证。核心依赖全部 MIT/BSD，底座仅需标准库 + PyYAML 即可运行。